MVC Conversion Tutorial in Express.js

Complete Breakdown of Converting a CRUD App into MVC Architecture

1. Overview

This tutorial explains how to convert an existing CRUD-based Express.js + MongoDB application into a clean, organized **MVC (Model–View–Controller)** architecture. The tutorial uses an existing Contact App and restructures the code for better maintainability, modularity, and scalability.

---

2. Understanding MVC Architecture

2.1 Meaning of MVC

- **Model:** Handles database schema, database queries, and data-related logic.
- **View:** Contains frontend template files (HTML/EJS).
- **Controller:** Contains business logic and acts as a mediator between the Model and View.

2.2 MVC Workflow

1. **User sends a request** (via route).
2. **Controller receives the request**.
3. Controller requests data from **Model**.
4. Model fetches data from **Database**.
5. Controller sends data to **View**.
6. View renders an HTML page with dynamic data.
7. Response is sent back to the user.

2.3 Benefits of MVC

- Organized and structured code
- Separation of concerns
- Reduced complexity
- Easy maintainability
- Independent modules
- Improved collaboration in teams
- Reusable code blocks
- Platform independent pattern

---

3. Existing Project Structure (Before MVC)

Initially, the entire CRUD app (CRUD operations + MongoDB connection + routes + Express setup) was coded inside a single file:

```
index.js
views/
models/
```

Problems:

- All routes in one file
- All controllers inside route definitions
- MongoDB connection also inside index.js
- Hard to update or scale
- Difficult to maintain as the app grows

---

4. Step-by-Step Conversion to MVC

4.1 Fixing View Issues Before MVC

Update Navigation Link

Change HTML link from:

```
href="index"
```

to

```
href="/"
```

Add Serial Number in EJS Table

Add counter variable:

```
<% let srno = 1; %>

<tbody>
<% contacts.forEach(contact => { %>
    <tr>
        <td><%= srno++ %></td>
        ...
    </tr>
<% }) %>
</tbody>
```

5. Creating MVC Folder Structure

```
project/
│
├── controllers/
│     └── contactController.js
│
├── models/
│     └── ContactModel.js
│
├── routes/
│     └── contactRoutes.js
│
├── views/      (already exists)
│
├── config/
│     └── database.js
│
├── index.js
└── .env
```

6. Switching from CommonJS to ES Modules

Update package.json

```
{
  "type": "module"
}
```

Replace `require()` with `import`

Example:

Old:

```
const express = require("express");
```

New:

```
import express from "express";
```

7. Moving Routes to Separate Files

Create `routes/contactRoutes.js`

```
import express from "express";
const router = express.Router();

// Example routes
router.get("/", getContacts);
router.get("/add", addContactPage);
router.post("/add", addContact);
router.get("/edit/:id", editContactPage);
router.post("/edit/:id", updateContact);
router.get("/delete/:id", deleteContact);

export default router;
```

Use this in index.js

```
import contactRoutes from "./routes/contactRoutes.js";

app.use("/", contactRoutes);
```

8. Moving Database Configuration to `/config/database.js`

New database configuration file

```
import mongoose from "mongoose";

const connectDB = async () => {
    await mongoose.connect(process.env.MONGO_URL);
};

export default connectDB;
```

Use in index.js

```
import connectDB from "./config/database.js";
connectDB();
```

9. Fixing Model Export/Import (ESM)

Update Model Export

```
export default Contact;
```

Update Model Import

```
import Contact from "../models/ContactModel.js";
```

10. Creating Controllers

Create `/controllers/contactController.js`

Example:

```
import Contact from "../models/ContactModel.js";

export const getContacts = async (req, res) => {
    const contacts = await Contact.find();
    res.render("home", { contacts });
};

export const getContact = async (req, res) => {
    const contact = await Contact.findById(req.params.id);
    res.render("singleView", { contact });
};

export const addContactPage = (req, res) => {
    res.render("addContact");
};

export const addContact = async (req, res) => {
    await Contact.create(req.body);
    res.redirect("/");
};

export const editContactPage = async (req, res) => {
```

```
    const contact = await Contact.findById(req.params.id);
    res.render("editContact", { contact });
};

export const updateContact = async (req, res) => {
    await Contact.findByIdAndUpdate(req.params.id, req.body);
    res.redirect("/");
};

export const deleteContact = async (req, res) => {
    await Contact.findByIdAndDelete(req.params.id);
    res.redirect("/");
};
```

Import these controller functions inside routes

```
import {
    getContacts,
    getContact,
    addContactPage,
    addContact,
    editContactPage,
    updateContact,
    deleteContact
} from "../controllers/contactController.js";
```

---

11. Using `.env` File for Configuration

Create `.env`

```
PORT=3000
MONGO_URL=mongodb://localhost:27017/contactdb
```

Install dotenv

```
npm install dotenv
```

Load environment variables where needed

In `database.js`

```
import dotenv from "dotenv";
dotenv.config();

await mongoose.connect(process.env.MONGO_URL);
```

In `index.js`

```
const port = process.env.PORT;
app.listen(port, () => console.log(`Server running on ${port}`));
```

---

12. Common Errors & Fixes Encountered

1. Missing `.js` extension

ESM requires full path:

```
import Contact from "../models/ContactModel.js";
```

2. Wrong folder paths

Use correct relative paths:

```
../models/
../controllers/
```

3. Missing imports

Ensure models are imported inside controllers, not routes.

4. Missing `dotenv` package

Install manually.

---

13. Final MVC Structure

```
controllers/
    contactController.js

models/
```

```
    ContactModel.js

routes/
    contactRoutes.js

config/
    database.js

views/
    *.ejs

.env
index.js
package.json
```
